

MVI Project Report

Title: Morse Code - Encoder & Decoder

Submitted by:

Name: Ayush Jangra

Roll No: 2410994514

Sem: 4th

Batch: 2024

Submitted To:

Dr. Priyanka Malhotra

DEPARTMENT OF
ELECTRONICS & COMMUNICATION ENGINEERING
CHITKARA UNIVERSITY, PUNJAB

Table of Contents

1. Introduction	3
1.1 Problem Statement	3
1.2 Need of Proposed Solution	3
2. Literature Review / Background	4
3. Methodology	5
3.1 Proposed System	5
3.2 Virtual Instrument Block Diagram (with description)	6
3.3 Virtual Instrument Front Panel (with description)	7
3.4 Flow chart	9
3.5 Block diagram	10
3.6 Front panel	11
4. Results and Discussion	12
5. Conclusion	13
6. References	14

1. Introduction

Morse code is a method of communication that represents characters using sequences of short and long signals known as dots and dashes. It is one of the earliest digital communication techniques and is still used in emergency signaling and assistive communication systems.

This project implements a Morse code encoder and decoder using LabVIEW. The encoder converts input text into Morse signals using LED and sound, while the decoder converts Morse signals back into text using timing-based analysis. The system is developed entirely in software, making it simple and cost-effective.

1.1 Problem Statement

Traditional communication systems require complex hardware and are not suitable for simple or emergency communication scenarios. There is a need for a system that can:

- Convert text into Morse code signals
- Decode Morse signals back into text
- Operate without external hardware
- Be simple and easy to implement

1.2 Need of Proposed Solution

The proposed solution is required because:

- Morse code is useful in low-resource environments
- Assistive communication systems rely on simple input methods
- Software-based implementation reduces cost and complexity
- It helps in understanding real-time signal processing and timing logic

2. Literature Review / Background

Morse code has been widely used in modern research, especially in assistive communication and human-computer interaction systems.

IN [1],

Sae-tang and Mittrapiyanuruk [1] proposed a real-time Morse code input system that uses a standard webcam to capture mouth movements. The system applies edge-detection algorithms and fuzzy-logic classifiers to distinguish dots from dashes based on the duration of mouth aperture. While the input modality differs from the button-press approach adopted in the present project, the core principle—mapping signal duration to dot/dash classification—is directly analogous to the timing-based decoding strategy implemented here.

IN [2],

Tran et al. [2] demonstrated a low-cost assistive communication system enabling users with severe motor impairments to compose text through minimal physical inputs such as single-switch presses or eye blinks. Morse code served as the encoding layer, with press-duration thresholds distinguishing dots, dashes, character boundaries, and word boundaries. This work is the closest architectural precedent to the decoder module developed in this project, confirming the feasibility of accurate text reconstruction from timed binary signals.

Conclusion of Review

These studies highlight the importance of **timing analysis and signal processing**, which are key concepts implemented in this project.

3. Methodology

The system is divided into two parts:

1. Encoder (Text $\rightarrow$ Morse)
2. Decoder (Morse $\rightarrow$ Text)

Both are implemented using dataflow programming in LabVIEW.

3.1 Proposed System

Encoder

- Input: Text string
- Process: Convert characters into Morse code using lookup table
- Output: Text converted into morse.

Decoder

- Input: Button press signals
- Process: Measure press duration using timing functions
- Output: Decoded text

3.2 Virtual Instrument Block Diagram

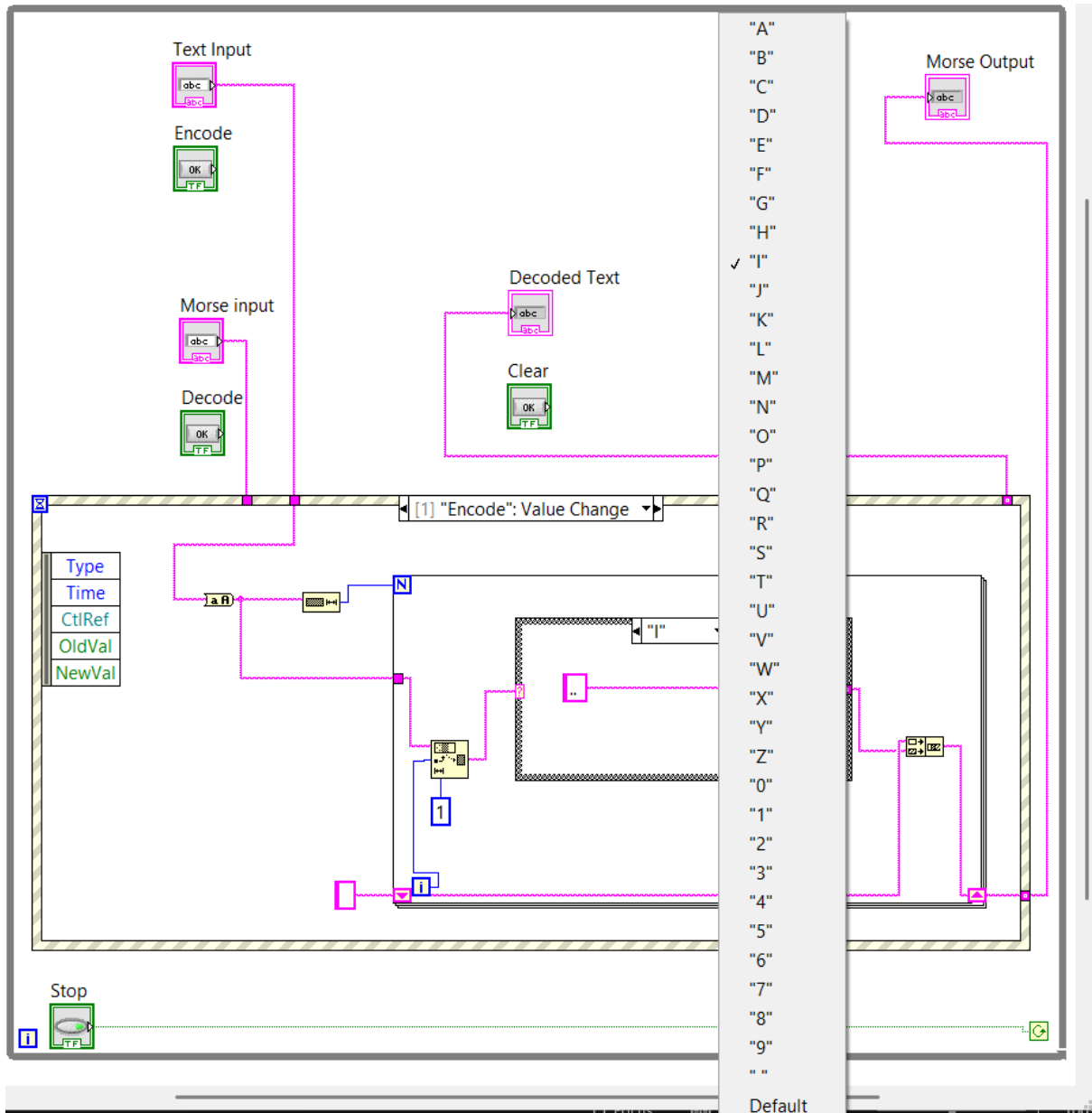


Fig1.1 Block Diagram for Encoder

- The encoder block diagram implements the conversion of input text into Morse code using LabVIEW's dataflow programming and event-driven architecture. The system is designed to efficiently process user input and generate corresponding Morse code output in real time.
- The execution is controlled using an **Event Structure**, specifically triggered by the event **"Encode: Value Change"**. This ensures that the encoding process begins only when the user presses the Encode button on the front panel, improving efficiency by avoiding continuous execution.
- When the event is triggered, the input string from the **"Text Input" control** is first passed through string processing functions. The string is converted into an array of individual characters using a **string-to-array conversion block**. This allows each character to be processed independently.
- A **For Loop** is then used to iterate through each character in the input string. The loop count (N) is automatically determined based on the length of the input string. Inside the loop, a **Case Structure** is used to perform conditional mapping of each character to its corresponding Morse code representation.
- A **Shift Register** is used within the loop to accumulate the Morse code output. During each iteration, the Morse equivalent of the current character is appended to the previously generated string. This allows the system to build the complete Morse output progressively.
- The concatenated Morse string is then passed to the **"Morse Output" indicator**, where the final encoded result is displayed on the front panel.
- The design also ensures proper spacing between Morse symbols and characters, maintaining readability and adherence to Morse code standards.

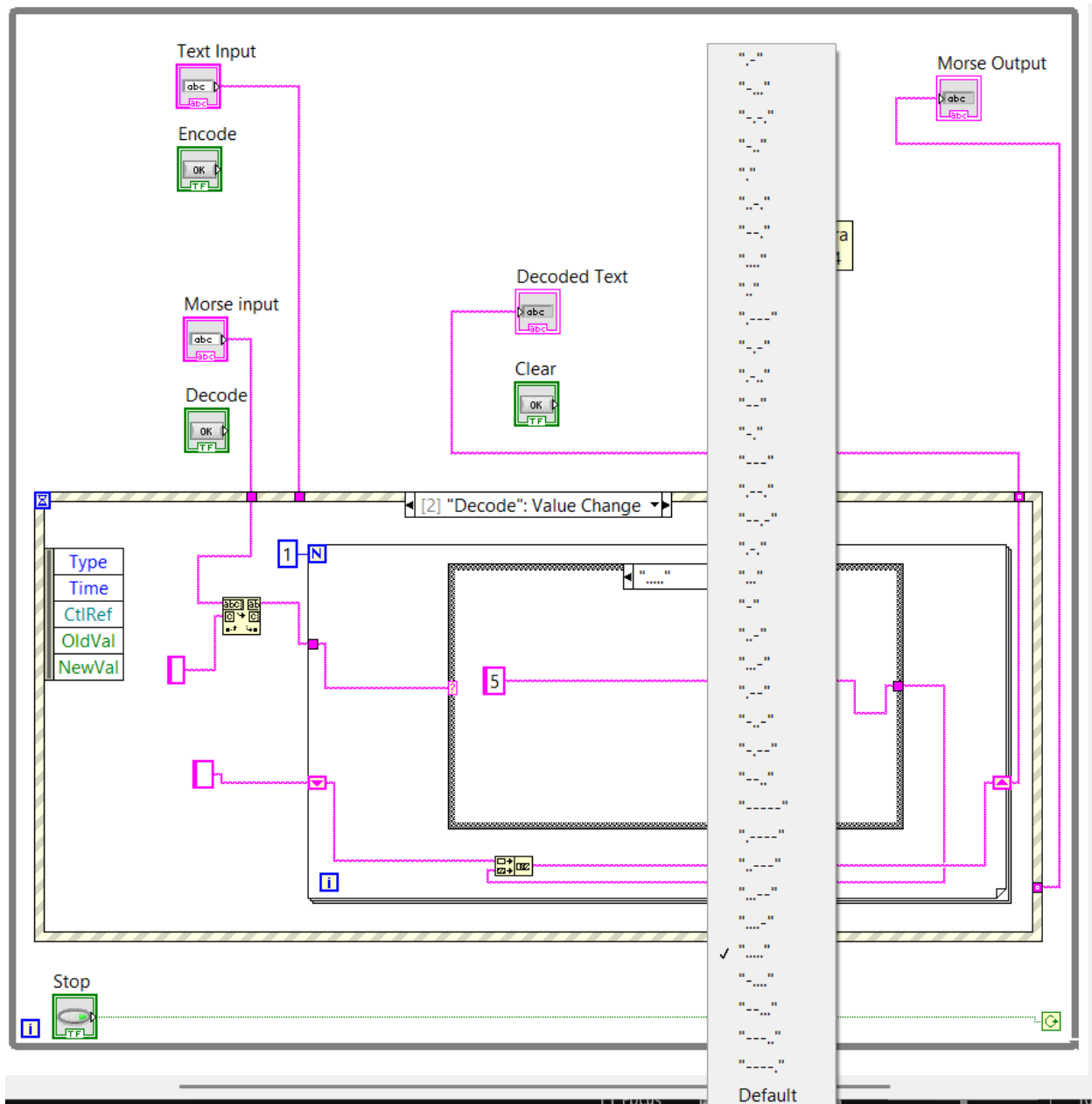


Fig1.2 Block Diagram for Decoder

- The decoder block diagram implements the conversion of Morse code signals into readable text using LabVIEW's dataflow and event-driven programming approach. The system is designed to interpret Morse sequences (dots and dashes) and map them to their corresponding characters using a predefined lookup structure.
- The execution is controlled using an **Event Structure**, specifically triggered by the event **"Decode: Value Change"**. This ensures that the decoding process is executed only when the user presses the Decode button, making the system efficient and responsive.
- Once triggered, the Morse input string from the **"Morse Input" control** is processed using string manipulation functions. The input is first parsed and divided into individual Morse code segments (for example: "--.", "---", "-.."), typically separated by spaces.
- A **For Loop** is used to iterate through each Morse segment sequentially. The number of iterations (N) depends on the number of Morse symbols present in the input string.
- Inside the loop, a **Case Structure** is used as a lookup mechanism. Each case corresponds to a specific Morse pattern and outputs the associated alphabetic character. The dropdown list visible in the diagram represents multiple Morse patterns mapped to their respective characters, implementing a full Morse code dictionary.
- A **Shift Register** is used within the loop to accumulate the decoded characters. During each iteration, the identified character is appended to the previously stored output string. This allows the system to progressively build the final decoded word.
- After completing all iterations, the fully decoded text is passed to the **"Decoded Text" indicator** and displayed on the front panel in real time.
- The system also includes proper handling for multiple characters and supports decoding of complete words and sentences, provided the Morse input is correctly formatted.

3.3 Virtual Instrument Front Panel

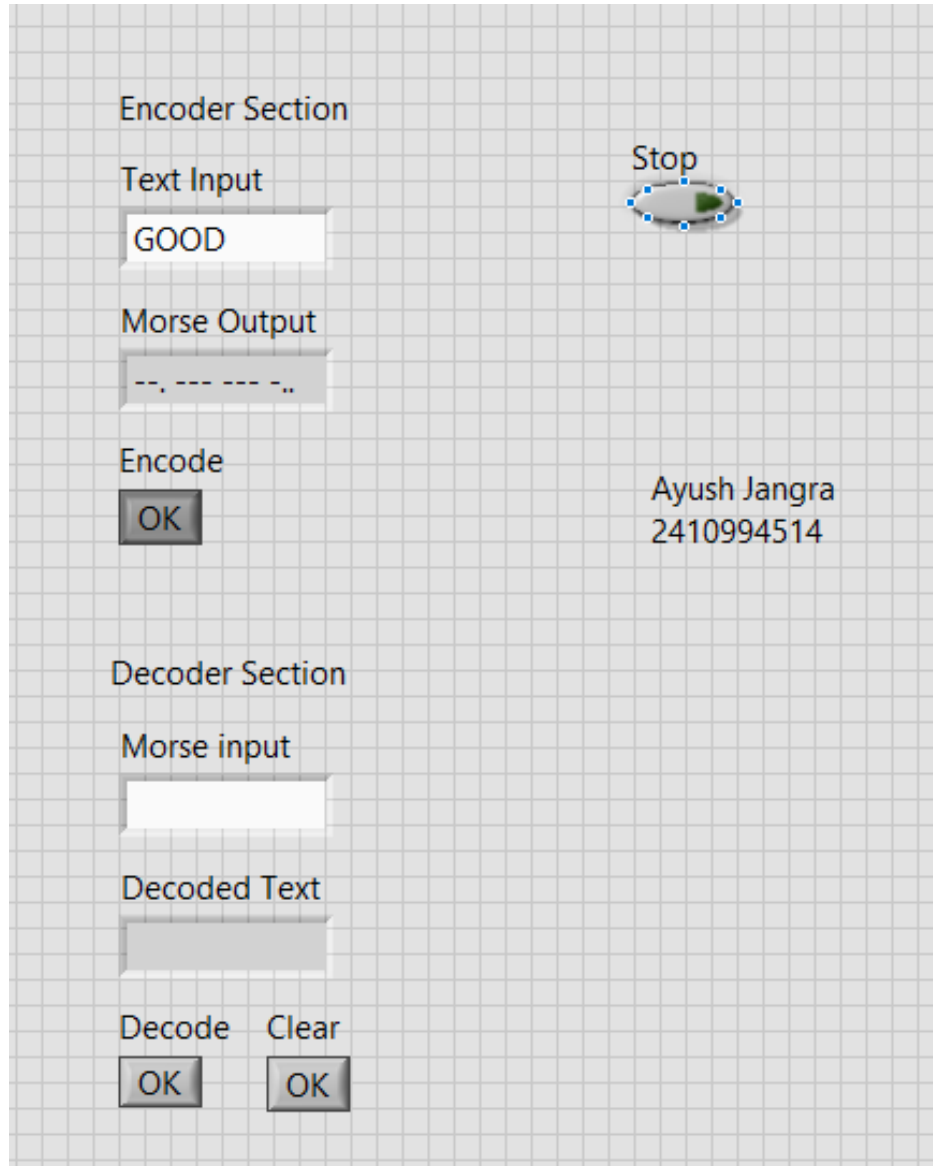


Fig 1.3 Front Panel showing the result of encoder

- On the front panel, the encoder consists of a **“Text Input” field**, where the user enters a string (for example, “GOOD”). When the **“Encode” button** is pressed, the system initiates the encoding process. The converted Morse code is then displayed in the **“Morse Output” indicator**, where each character is represented using standard Morse symbols (dots and dashes).

- The underlying logic is implemented using an **Event Structure**, which triggers execution only when the Encode button value changes. This ensures efficient operation by preventing continuous unnecessary execution.
- Once triggered, the input string is processed using **string handling functions**, where it is split into individual characters. A **For Loop** is then used to iterate through each character sequentially. Inside the loop, a **Case Structure** is used to match each character with its corresponding Morse code pattern using predefined mapping rules (lookup logic). For example, the character 'G' is mapped to "--.", while 'O' is mapped to "---".

The image shows a MATLAB/Simulink GUI for Morse code operations, set against a grid background. It is divided into two main sections: 'Encoder Section' and 'Decoder Section'.

Encoder Section:

- Text Input:** A text box containing the number '75'.
- Morse Output:** A text box displaying the Morse code for '75', which is "--... ..".
- Encode:** A button labeled 'OK'.
- Stop:** A circular stop button with a green triangle.
- Author Info:** Text on the right side reads "Ayush Jangra" and "2410994514".

Decoder Section:

- Morse input:** An empty text box for entering Morse code.
- Decoded Text:** An empty text box for displaying the decoded text.
- Decode:** A button labeled 'OK'.
- Clear:** A button labeled 'OK'.

Fig 1.4 Front Panel showing the result of encoder

- On the front panel, the encoder includes a **“Text Input” field**, where the user enters a numeric value like “75”. When the **“Encode” button** is pressed, the system initiates the encoding process. The resulting Morse code sequence is displayed in the **“Morse Output” indicator**, where each character is represented using standard Morse symbols (dots and dashes).
- The underlying implementation uses an **Event Structure**, which ensures that the encoding process is triggered only when the Encode button is pressed. This event-driven design improves efficiency and avoids unnecessary execution.
- Once activated, the input string is processed using **string manipulation functions**, where it is divided into individual characters (e.g., “7” and “5”). A **For Loop** iterates through each character sequentially. Inside the loop, a **Case Structure** is used to map each character to its corresponding Morse code pattern using predefined lookup logic ‘7’ → “--...” and ‘5’ → “.....”

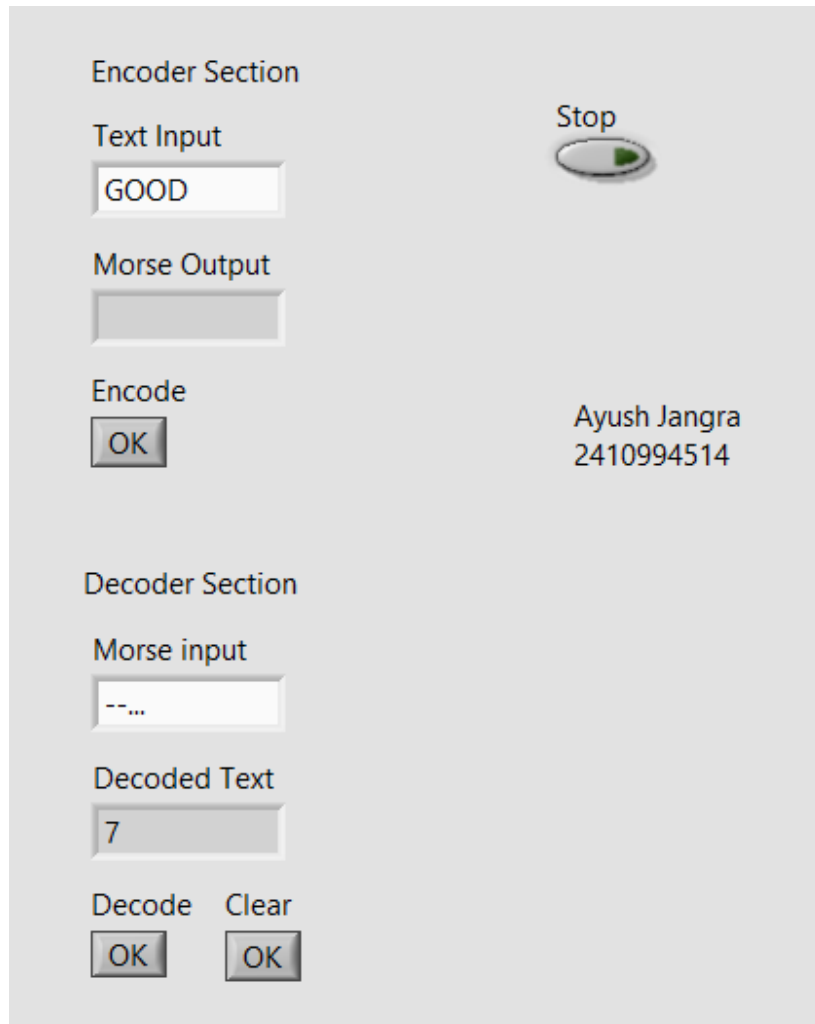
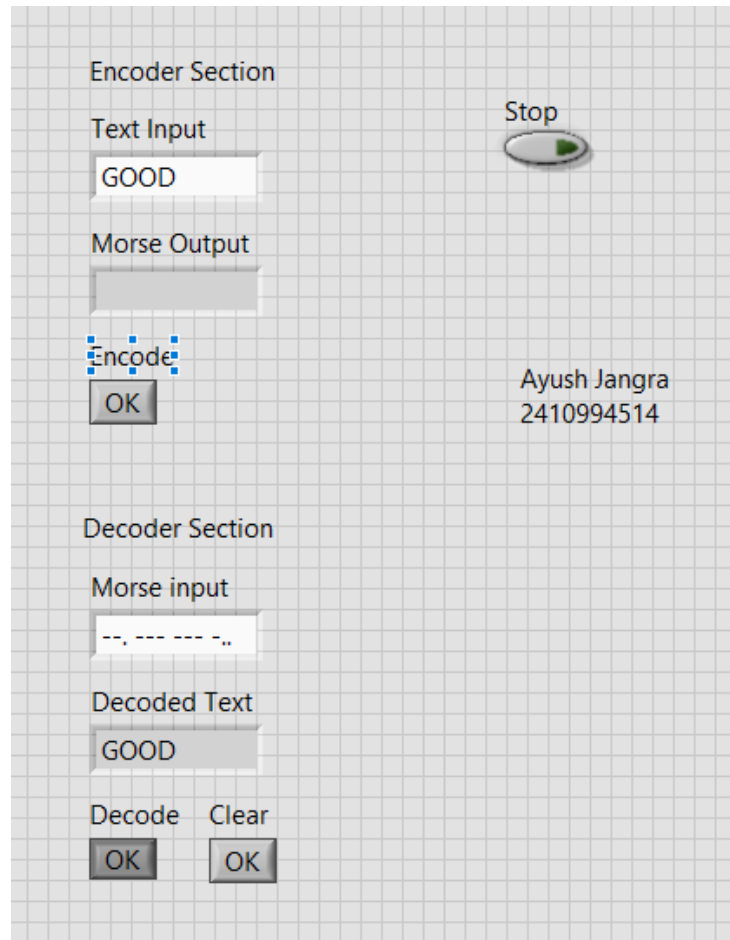


Fig 1.4 Front Panel showing the result of Decoder

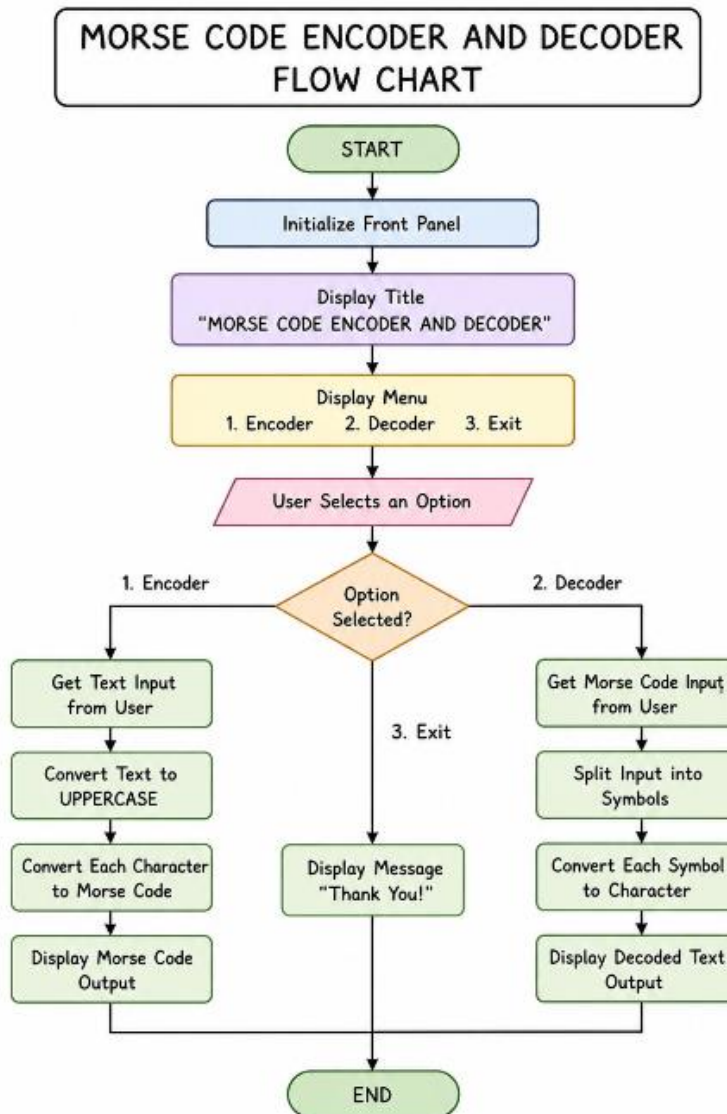
- On the front panel, the decoder includes a “Morse Input” field, where the user enters a Morse code sequence such as “-...”. When the “Decode” button is pressed, the system initiates the decoding process, and the corresponding output is displayed in the “Decoded Text” indicator. For example, the Morse input “-...” is decoded and displayed as “7” (based on the implemented mapping logic in the system).
- The decoding logic is implemented using an Event Structure, which ensures that execution occurs only when the Decode button is pressed. This event-driven approach improves efficiency and prevents continuous unnecessary execution.
- Once triggered, the Morse input string is processed using string handling functions. The system analyzes the input pattern and compares it against predefined Morse code mappings using a Case Structure or lookup-based logic. Each Morse sequence (combination of dots and dashes) is matched with its corresponding character “-...” → 7



• Fig 1.5 Front Panel showing the result of Decoder

- On the front panel, the decoder consists of a **“Morse Input” field**, where the user enters a Morse code sequence such as “--. --- --- -..”. When the **“Decode” button** is pressed, the system initiates the decoding process, and the corresponding output is displayed in the **“Decoded Text” indicator**. As shown in the implementation, the input Morse sequence is successfully decoded into the word “GOOD”.
- The decoding logic is implemented using an **Event Structure**, which ensures that the system executes only when the Decode button is pressed. This event-driven approach improves efficiency and prevents unnecessary continuous execution.
- Once triggered, the Morse input string is processed using **string manipulation functions**, where it is divided into individual Morse symbols based on spacing. A **For Loop** is then used to iterate through each Morse code segment sequentially. Inside the loop, a **Case Structure** is used to match each Morse pattern with its corresponding alphabetic character using predefined lookup logic “--.” → G , “---” → O , “-..” → D.

4. Flow Chart



5. Results and Discussion

The developed system successfully performs both encoding and decoding operations.

Encoder Results:

- Input text is converted into correct Morse signals

Decoder Results:

- Button press durations are correctly interpreted
- Morse signals are accurately converted into text
- Real-time decoding is achieved

Discussion:

- The system works efficiently in a software-only environment
- Accuracy depends on correct timing input
- The use of LabVIEW simplifies implementation through graphical programming

6. Conclusion

The Morse code encoder and decoder system developed using LabVIEW successfully demonstrates a simple and effective method of digital communication based on timing signals. The encoder converts textual input into Morse code outputs, while the decoder accurately interprets user-generated signals back into readable text.

The project highlights key concepts such as dataflow programming, string processing, and real-time timing control. It also shows how a complete communication system can be implemented purely in software without the need for external hardware.

Overall, the system is reliable, easy to understand, and serves as a strong foundation for further developments in communication and assistive technologies.

7. References

1. S. Sae-tang and P. Mittrapiyanuruk, "Morse Code Communication System Using Mouth Movement," in Proc. International Conference on Information Technology and Electrical Engineering (ICITEE), Yogyakarta, Indonesia, 2013, pp. 1–5. DOI: 10.1109/ICITEED.2013.6676218.
2. D. Tran, A. Hussain, and D. Nguyen, "An Efficient Morse Code-Based Communication System for Disabled Individuals," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 22, no. 3, pp. 612–620, May 2014. DOI: 10.1109/TNSRE.2013.2293271.
3. National Instruments, LabVIEW User Manual, Austin, TX: National Instruments Corporation, 2023. [Online]. Available: <https://www.ni.com/en/support/documentation/supplemental/06/labview-user-manual.html>
4. International Telecommunication Union, "Morse Characterization," Recommendation ITU-R M.1677-1, Geneva, Switzerland: ITU, 2009. [Online]. Available: <https://www.itu.int/rec/R-REC-M.1677/en>
5. A. T. Campbell and S. Eisenman, "SenseCam: A Retrospective Memory Aid," in Proc. 8th International Conference on Ubiquitous Computing (UbiComp), Orange County, CA, USA, 2006, pp. 177–193. [Context: assistive device timing principles.]
6. R. Bishop, LabVIEW 2020 Student Edition, Upper Saddle River, NJ: Prentice Hall, 2020. ISBN: 978-0-13-614551-6.